Problem 1 :

<u>Initial</u> **Vertex- Distance, Previous** Anya - ∞, null Frog - ∞, null Dog - ∞, null Penguin - ∞, null Orca - ∞, null Rabbit - ∞, null Wolf - ∞, null Elephant - ∞, null Rhino - ∞, null Bond - ∞, null Panda - ∞, null Sloth - ∞, null **<u>toVisit:</u>** Anya (Since she is the start) this is a priority queue on minimum. **<u>visited:</u>**	<u>Visit Anya</u>, she's start so distance = 0. Add all her vertices from her outgoing edges to toVisit, and calculate their distances with Anya as the previous. Frog dist = weight + prevDist = 0.1+0 = 0.1; 0.1 < ∞ so update dist and previous to Anya. Dog dist = weight + prevDist = 4 + 0 = 4; 4 < ∞ so update dist and previous to Anya Penguin dist = weight + prevDist = 2 + 0 = 2; 2 < ∞ so update dist and previous to Anya. Orca dist = weight + prevDist = 1 + 0 = 1; 1 < ∞ so update dist and previous to Anya. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - ∞, null Elephant - ∞, null Rhino - ∞, null Bond - ∞, null Panda - ∞, null Sloth - ∞, null Add Anya to visited. **<u>toVisit:</u>** Frog, Dog, Penguin, Orca **<u>visited:</u>** Anya

Visit Frog, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and update their distances and pointers, but it has none so no updates. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - ∞, null Elephant - ∞, null Rhino - ∞, null Bond - ∞, null Panda - ∞, null Sloth - ∞, null Add Frog to visited. **toVisit:** Dog, Penguin, Orca **visited:** Anya, Frog	Visit Orca, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and cacluate their distances with Orca as the previous, which is Wolf, Rhino, and Elephant. Wolf dist = weight + prevDist = 5 + 1 = 6; 6 < inf so update dist and previous to Orca. Rhino dist = weight + prevDist = 1 + 1 = 2; 2 < inf so update dist and previous to Orca. Elephant = weight + prevDist = 3 + 1 = 4; 4 < inf so update dist and previous to Orca. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - 6, Orca Elephant - 4, Orca Rhino - 2, Orca Bond - ∞, null Panda - ∞, null Sloth - ∞, null Add Orca to visited **toVisit:** Dog, Penguin, Wolf, Rhino, Elephant **visited:** Anya, Frog, Orca
Visit Penguin, because it is tied with the lowest priority in toVisit with Rhino so this was randomly chosen. Add all its vertices from outgoing edges to toVisit, and calculate	Visit Rhino, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances

their distances with Penguin as previous. Dog already in toVisit so its not added again. Dog dist = weight + prevDist = 2 + 2 = 4; 4 < 4 is not true so the distance and previous for Dog is not updated. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - 6, Orca Elephant - 4, Orca Rhino - 2, Orca Bond - ∞, null Panda - ∞, null Sloth - ∞, null Add penguin to visited **<u>toVisit:</u>** Dog, Wolf, Rhino, Elephant **<u>visited:</u>** Anya, Frog, Orca, Penguin	with Rhino as previous. Elephant is in toVisit so its not added again. Elephant dist = weight + prevDist = 0.5 + 2 = 2.5; 2.5 < 4 so distance and previous are updated to be Rhino and 2.5. Panda dist = weight + prevDist = 12 + 2 = 14; 14 < ∞ so distance and previous are updated. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - ∞, null Panda - 14, Rhino Sloth - ∞, null Add Rhino to visited **<u>toVisit:</u>** Dog, Wolf, Elephant, Panda **<u>visited:</u>** Anya, Frog, Orca, Penguin, Rhino
<u>Visit Elephant</u>, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Elephant as previous, which is Peenguin. Penguin has been visited so its not added again.	<u>Visit Dog</u>, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Dog as previous, which is Rabbit. Rabbit dist = weight + prevDist = 3 + 4 = 7; 7 < ∞ so distance and previous are updated. **Vertex - Distance, Previous** Anya - 0, null

Penguin dist = weight + prevDist = 3 + 2.5 = 5.5; 5.5 < 2 is not true, so distance and previous are not updated. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - ∞, null Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - ∞, null Panda - 14, Rhino Sloth - ∞, null Add Elephant to visited **toVisit:** Dog, Wolf, Panda **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant	Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - ∞, null Panda - 14, Rhino Sloth - ∞, null Add Dog to visited **toVisit:** Wolf, Panda, Rabbit **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog
Visit Wolf, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Wolf as previous. It has none so nothing is updated. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya	Visit Rabbit, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Rabbit as previous, which are Frog and Bond. Frog has been visited so it's not added to toVisit. Frog dist = weight + prevDist = 2 + 7 = 9; 9 < 0.1 is not true so distance and previous are not updated. Bond dist = weight + prevDist = 5 + 7 = 12; 12 < ∞ so distance and previous are updated.

Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - ∞, null Panda - 14, Rhino Sloth - ∞, null Add Wolf to visited **<u>toVisit:</u>** Panda, Rabbit **<u>visited:</u>** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf	**Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - 12, Rabbit Panda - 14, Rhino Sloth - ∞, null Add Rabbit to visited **<u>toVisit:</u>** Panda, Bond **<u>visited:</u>** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit
<u>Visit Bond</u>, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Bond as previous, which are Dog and Panda. Dog has been visited so it's not added to toVisit, and Panda is already in toVisit so it's not added again. Dog dist = weight + prevDist = 1 + 12 = 13; 13 < 4 is not true so distance and previous are not updated. Panda dist = weight + prevDist = 1.5 + 12 = 13.5; 13.5 < 14 so distance and previous are updated.	<u>Visit Panda</u>, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Panda as previous, which are Bond and Sloth. Bond has been visited so it's not added to toVisit. Bond dist = weight + prevDist = 1.7 + 13.5 = 15.2; 15.4 < 13.5 is not true so distance and previous are not updated. Sloth dist = weight + prevDist = 0.5 + 13.5 = 14; 14 < ∞ so distance and previous are updated. **Vertex - Distance, Previous**

Vertex - Distance, Previous Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - 12, Rabbit Panda – 13.5, Bond Sloth - ∞, null Add Bond to visited **toVisit:** Panda **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond	Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - 12, Rabbit Panda – 13.5, Bond Sloth - 14, Panda Add Panda to visited **toVisit:** Sloth **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond, Panda
Visit Sloth, because it has the lowest priority in toVisit. Add all its vertices from outgoing edges to toVisit, and calculate their distances with Sloth as previous, which are Elephant and Wolf. Both have been visited so they are not added to toVisit. Elephant dist = weight + prevDist = 1 + 14 = 15; 15 < 2.5 is not true so distance and previous are not updated. Wolf dist = weight + prevDist = 1 + 14 = 15; 15 < 6 is not true so distance and previous are not updated. **Vertex - Distance, Previous** Anya - 0, null	ToVisit is empty so the iterations are done and the shortest path to each vertex from Anya has been found. **Vertex - Distance, Previous** Anya - 0, null Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - 12, Rabbit

Frog – 0.1, Anya Dog - 4, Anya Penguin - 2, Anya Orca - 1, Anya Rabbit - 7, Dog Wolf - 6, Orca Elephant – 2.5, Rhino Rhino - 2, Orca Bond - 12, Rabbit Panda – 13.5, Bond Sloth - 14, Panda Add Sloth to visited **toVisit:** **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond, Panda, Sloth	Panda – 13.5, Bond Sloth - 14, Panda Add Sloth to visited **toVisit:** **visited:** Anya, Frog, Orca, Penguin, Rhino, Elephant, Dog, Wolf, Rabbit, Bond, Panda, Sloth

The shortest path from Anya to each vertex is:

Anya - Anya

Frog – Anya, Frog

Dog – Anya, Dog

Penguin – Anya, Penguin

Orca – Anya, Orca

Rabbit – Anya, Dog, Rabbit

Wolf - Anya, Orca, Wolf

Elephant – Anya, Orca, Rhino, Elephant

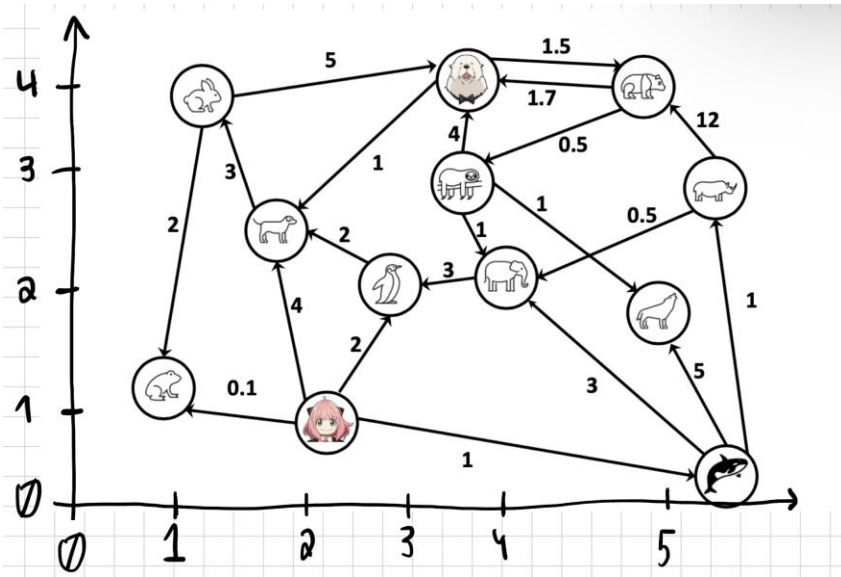
Rhino – Anya, Orca, Rhino

Bond – Anya, Dog, Rabbit, Bond

Panda – Anya, Dog, Rabbit, Bond, Panda

Sloth - Anya, Dog, Rabbit, Bond, Panda, Sloth

Problem 2:



For this A*, g(x) is the weight of the edge + the g of the previous vertex and h(x) is the Manhattan distance = |x_start – x_destination| + |y_start – y_destination|. For that graph, there are axes made to inidcate the x and y position, but to clarify at x = 1, there is Frog and Rabbit, at x =2 there is Anya and Dog, x = 3 has Penguin, x = 4 has Bond, Dog, and Sloth, and x = 5 has Panda, Rhino, Wolf, and Orca. At y = 0 there is Orca, y = 1 has Frog and Anya, y = 2 has Penguin, Elephant, and Wolf, y = 3 has Dog, Sloth, and Rhino, and y = 4 has Rabbit, Bond, and Panda.

To visit is a priority queue for minimum distance.

<u>Initial</u> **Vertex – F, G, H, Previous** Anya - ∞, ∞, ∞, null Frog - ∞, ∞, ∞, null Dog - ∞, ∞, ∞, null Penguin - ∞, ∞, ∞, null Orca - ∞, ∞, ∞, null Rabbit - ∞, ∞, ∞, null	<u>Visit Anya</u> since it has lowest priority Anya is the starting point, so f, g, and h are 0 and previous is null, and find the distance for all her adjacent vertices, being Frog, Dog, Penguin, and Orca, with their previous. Also add them to toVisit. Frog f = g(x) + h(x) = 0.1 + |2– 1|+|1-1| = 0.1 + |1|+|0| = 0.1 + 1 = 1.1; 1.1 < ∞ so previous, f, g, and h are updated.

Wolf - ∞, ∞, ∞, null Elephant - ∞, ∞, ∞, null Rhino - ∞, ∞, ∞, null Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null **toVisit: Anya** **Visited:**	Dog f = g(x) + h(x) = 4 + |2– 2|+|1-3| = 4+ |0| + |-2| = 4 + 0+2 = 4+ 2 = 6. 6 < ∞ so previous, f, g, and h are updated. Penguin f = g(x) + h(x) = 2 + |2-3| + |1-2| = 2 + |-1| + |-1| = 2 + 1 + 1 = 2 + 2 = 4. 4 < ∞ so previous, f, g, and h are updated. Orca f = g(x) + h(x) = 1 + |2-5| + |1-0| = 1 + |-3| + |1| = 1 + 3 + 1 = 1+4 = 5. 5 < ∞ so previous, f, g, and h are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - ∞, ∞, ∞, null Wolf - ∞, ∞, ∞, null Elephant - ∞, ∞, ∞, null Rhino - ∞, ∞, ∞, null Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null Add Anya to visited. **toVisit:** Frog, Dog, Penguin, Orca **Visited:** Anya
<u>Visit Frog</u> since it has lowest priority Anya is the starting point. Find the distance for Frog's adjacent vertices, which is nothing since it has no outgoing edges, so nothing is updated or added to toVisit. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya	<u>Visit Penguin</u> since it has lowest priority Anya is the starting point. Find the distance for Penguin's adjacent vertices, which is Dog. Dog is in toVisit already, so it is not added again. Dog f = g(x) + h(x) = (2 + 2)+ (|2– 2|+|1-3|) = 4+ (|0| + |-2|) = 4 + (0+2) = 4+ 2 = 6. 6 < 6, is false so nothing is updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya

Orca - 5, 1, 4, Anya Rabbit - ∞, ∞, ∞, null Wolf - ∞, ∞, ∞, null Elephant - ∞, ∞, ∞, null Rhino - ∞, ∞, ∞, null Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null Add Frog to visited. **toVisit:** Dog, Penguin, Orca **Visited:** Anya, Frog	Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - ∞, ∞, ∞, null Wolf - ∞, ∞, ∞, null Elephant - ∞, ∞, ∞, null Rhino - ∞, ∞, ∞, null Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null Add Penguin to visited. **toVisit:** Dog, Orca **Visited:** Anya, Frog, Penguin
Visit Orca since it has lowest priority Anya is the starting point. Find the distance for Orca's adjacent vertices, which are Elephant, Wolf, and Rhino. Then add all of them to toVisit. Elephant f = g(x) + h(x) = (3 + 1)+ (|2– 4|+|1-2|) = 4+ (|-2| + |-1|) = 4 + (2+1) = 4+ 3 = 7. 7 < ∞, so previous, f, g, and g are updated. Wolf f = g(x) + h(x) = (5 + 1)+ (|2– 5|+|1-2|) = 6+ (|-3| + |-1|) = 6 + (3+1) = 6+ 4 = 10. 10 < ∞, so previous, f, g, and g are updated. Rhino f = g(x) + h(x) = (1 + 1)+ (|2– 5|+|1-3|) = 2+ (|-3| + |-2|) = 2 + (3+2) = 2 + 5 = 7. 7 < ∞, so previous, f, g, and g are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - ∞, ∞, ∞, null Wolf - 10, 6, 4, Orca	Visit Dog since it has lowest priority Anya is the starting point. Find the distance for Dog's adjacent vertices, which is Rabbit. Then add it to toVisit. Rabbit f = g(x) + h(x) = (3 + 4)+ (|2– 1|+|1-4|) = 7+ (|1| + |-3|) = 7 + (1+3) = 7 + 4 = 11. 11 < ∞, so previous, f, g, and g are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant - 7, 4, 3, Orca Rhino - 7, 2, 5, Orca Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null

Elephant - 7, 4, 3, Orca Rhino - 7, 2, 5, Orca Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null Add Orca to visited. **toVisit:** Dog, Elephant, Wolf, Rhino **Visited:** Anya, Frog, Penguin, Orca	Add Dog to visited. **toVisit:** Elephant, Wolf, Rhino, Rabbit **Visited:** Anya, Frog, Penguin, Orca, Dog
<u>Visit Elephant</u> since it has lowest priority Anya is the starting point. It is tied with Rhino for lowest f, but it has the lower h so it is chosen. Find the distance for Elephant's adjacent vertices, which is Penguin, which is already visited so its not added to toVisit. Penguin f = g(x) + h(x) = (3 + 4)+ (|2– 3|+|1-2|) = 7+ (|-1| + |-1|) = 7 + (1+1) = 7 + 2 = 9. 9 < 4, is false so nothing is updated for Penguin. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant - 7, 4, 3, Orca Rhino - 7, 2, 5, Orca Bond - ∞, ∞, ∞, null Panda - ∞, ∞, ∞, null Sloth - ∞, ∞, ∞, null Add Elephant to visited. **toVisit:** Wolf, Rhino, Rabbit	<u>Visit Rhino</u> since it has lowest priority Anya is the starting point. Find the distance for Rhino's adjacent vertices, which is Elephant, which is already visited so its not added to toVisit, and Panda. Add Panda to toVisit. Elephant f = g(x) + h(x) = (0.5 + 2)+ (|2–4|+|1-2|) = 2.5 + (|-2| + |-1|) = 2.5 + (2+1) = 2.5 + 3 = 5.5. 5.5 < 7, so f, g, h, and previous are updated for Elephant. Panda f = g(x) + h(x) = (12 + 2) + (|2-5|+|1-4|) = 14 + (|-3| + |-3|) = 14 + (3+3) = 14 + 6 = 20. 20 < ∞, so previous, f, g, and h for Panda is updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - ∞, ∞, ∞, null Panda - 20, 14, 6, Rhino Sloth - ∞, ∞, ∞, null

Visited: Anya, Frog, Penguin, Orca, Dog, Elephant	Add Rhino to visited. **toVisit:** Wolf, Rabbit, Panda **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino
<u>Visit Wolf</u> since it has lowest priority Anya is the starting point. Find the distance for Wolf's adjacent vertices, which is Nothing, so nothing is changed. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - ∞, ∞, ∞, null Panda - 20, 14, 6, Rhino Sloth - ∞, ∞, ∞, null Add Wolf to visited. **toVisit:**Rabbit, Panda **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf	<u>Visit Rabbit</u> since it has lowest priority Anya is the starting point. Find the distance for Wolf's adjacent vertices, which are Frog and Bond, and add them to toVisit, Frog has been visited so its not added. Frog f = g(x) + h(x) = (2 + 7) + (|2-1| + |1-1|) = 9 + (|1| + |0|) = 9 + (1+0) = 9 + 1 = 10 < 1.1 is not true so nothing is updated for Frog. Bond f = g(x) + h(x) = (5 + 7) + (|2-4| + |1-4|) = 12 + (|-2| + |-3|) = 12 + (2+3) = 12 + 5 = 17. 17 < ∞ so f, g, h, and previous for bond are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - 17, 12, 5, Rabbit Panda - 20, 14, 6, Rhino Sloth - ∞, ∞, ∞, null Add Rabbit to visited. **toVisit:** Panda, Bond **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf, Rabbit
<u>Visit Bond</u> since it has lowest priority	<u>Visit Panda</u> since it has lowest priority

Anya is the starting point. Find the distance for Bond's adjacent vertices, which are Dog and Panda. Dog has been visited so its not added to toVisit, and Panda is already in there so its not added again. Dog f = g(x) + h(x) = (1 + 12) + (|2-2| + |1-3|) = 13 + (|0| + |-2|) = 13 + (0+2) = 13 + 2 = 15 < 6 is not true so nothing is updated for Dog. Panda f = g(x) + h(x) = (1.5 + 12) + (|2-5| + |1-4|) = 13.5 + (|-3| + |-3|) = 13.5 + (3+3) = 13.5 + 6 = 19.5. 19.5 < 20 so f, g, h, and previous for Panda are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - 17, 12, 5, Rabbit Panda – 19.5, 13.5, 6, Bond Sloth - ∞, ∞, ∞, null Add Bond to visited. **toVisit:** Panda **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf, Rabbit, Bond	Anya is the starting point. Find the distance for Panda's adjacent vertices, which are Bond and Sloth. Bond has been visited so its not added to toVisit and Sloth is added. Bond f = g(x) + h(x) = (1.7 + 13.5) + (|2-4| + |1-4|) = 15.2 + (|-2| + |-3|) = 15.2 + (2+3) = 15.2 + 5 = 20.2 <17 is not true so nothing is updated for Bond. Sloth f = g(x) + h(x) = (0.5 + 13.5) + (|2-4| + |1-3|) = 14 + (|-2| + |-2|) = 14 + (2+2) = 14 + 4 = 18. 18 < ∞ so f, g, h, and previous for Sloth are updated. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - 17, 12, 5, Rabbit Panda – 19.5, 13.5, 6, Bond Sloth - 18, 14, 4, Panda Add Panda to visited. **toVisit:** Sloth **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf, Rabbit, Bond, Panda
<u>Visit Sloth</u> since it has lowest priority Anya is the starting point. Find the distance for Sloth's adjacent vertices, which are Bond, Elephant, and Wolf. They all have been visited so they are not added to toVisit. Bond f = g(x) + h(x) = (4 + 14) + (|2-4| + |1-4|) = 18 + (|-2| + |-3|) = 18 + (2+3) = 18 + 5 = 23 <	<u>toVisit is empty to loop ends.</u> That means the shortest distance and path to every vertex from Anya has been found. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya

17 is not true so nothing is updated for Bond. Elephant f = g(x) + h(x) = (1 + 14) + (|2-4| + |1-2|) = 18 + (|-2| + |-1|) = 18 + (2+1) = 18 + 3 = 21 < 5.5 is not true so nothing is updated for Elephant. Wolf f = g(x) + h(x) = (1 + 14) + (|2-5| + |1-2|) = 18 + (|-3| + |-1|) = 18 + (3+1) = 18 + 4 = 22 < 10 is not true so nothing is updated for Wolf. **Vertex – F, G, H, Previous** Anya - 0, 0, 0, null Frog – 1.1, 0.1, 1, Anya Dog - 6, 4, 2, Anya Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - 17, 12, 5, Rabbit Panda – 19.5, 13.5, 6, Bond Sloth - 18, 14, 4, Panda Add Sloth to visited. **toVisit:** **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf, Rabbit, Bond, Panda, Sloth.	Penguin - 4, 2, 2, Anya Orca - 5, 1, 4, Anya Rabbit - 11, 7, 4, Dog Wolf - 10, 6, 4, Orca Elephant – 5.5, 2.5, 3, Rhino Rhino - 7, 2, 5, Orca Bond - 17, 12, 5, Rabbit Panda – 19.5, 13.5, 6, Bond Sloth - 18, 14, 4, Panda **toVisit:** **Visited:** Anya, Frog, Penguin, Orca, Dog, Elephant, Rhino, Wolf, Rabbit, Bond, Panda, Sloth.

The shortest path from Anya to every vertex is:

Anya - Anya

Frog – Anya, Frog

Dog – Anya, Dog

Penguin – Anya, Penguin

Orca - Anya, Orca

Rabbit – Anya, Dog, Rabbit

Wolf – Anya, Orca, Wolf

Elephant – Anya, Orca, Rhino, Elephant

Rhino - Anya, Orca, Rhino

Bond - Anya, Dog, Rabbit, Bond

Panda – Anya, Dog, Rabbit, Bond, Panda

Sloth – Anya, Dog, Rabbit, Bond, Panda, Sloth

Problem 3:

If H is referring to which found the shortest path from Anya to all vertices in the least iterations, then they are tied. This is because Dijkstra found the shortest path to each vertices in 12 iterations, while A* found them all in 12 too. The reason they are both 12 is because they must visit every single vertex to find the shortest path to it. The only difference is that the order of nodes visited is not the same for both in this case. If H is referring to the deepest vertex which is sloth, then the answer is still the same because for both methods the deepest one is usually the last vertex visited, which means it will need to visit 11 nodes before the last one. If H is refereeing to a node like Orca than Dijkstra is faster, but if it is Penguin then A* is faster. The reason for this is because Dijkstra will only visit vertices based off which has the shortest distance so far, which is only determined by the edge weights, but A* visits vertices based off the sum of edge weight and potential distance from Anya. This leads to Dijkstra being able to reach vertices that are geographically farther from Anya quicker than A* if it has a short distance, but A* looks at ones that have a combination of the shortest distance and closes geographically leading it to visit vertices like Dog and Penguin earlier.

Problem 7:

<u>Problem 4:</u>

The time complexity of this algorithm is O(n^2) where n is the length of the inputted adjacency matrix. This is because the algorithm first initializes a variable to be the matrix's length with .length. .length on the matrix is a constant time operation since it returns the

size of the array, which is a constant time operation. This means that the first initialization is O(1). Then after that is an if statement that is not taken because it would return early and result in less operations, but the condition is still evaluated and only has one comparison, which is just a constant time operation, being O(1). After that a LinkedList is created and initialized, which has a time complexity of O(1) since it just making an empty linkedList. Following that, a Boolean array is created and initialized to be of size n, which means that it must set all those n values to false, which makes this O(n). After than another variables is initialized to 0, which is just a constant time operation being O(1). After that, 0 is added to the front of the linked list, which is a constant time operation since it is just creating a new and updating the head pointer to that, all of which are constant time operations making adding O(1). Following that is an array update, arithmetic, and a variable initializations, which are constant time operations being O(1). Then there is a while loop, with a condition that call on isEmpty for a linked list, which is a constant time operation since it just does a comparison to see if the size is 0, meaning the while loop's comparison is O(1). The loop's body first has an update and method call to remove for the linked list, which both are constant time operations, since remove just returns a value and updates pointers being O(1). Following that is a for loop, which has a body that has an if segment with a condition that does array retrievals and comparisons, making that be O(1). The if segment's body has arithmetic, updating a value in an array, and adding a value to the front of the linked list. All of these are constant time operations and adding to the front of a linked list with addFirst is also constant time since it just creates a new node and updates pointer which are constant time operations, meaning the body of the if segment is O(1). Since both the body and condition of the if segment is O(1), the if segment as a whole is O(1+1) = O(1) since constants are dropped. The for loop itself initializes a variable once, and then does a comparison and arithmetic, but those two happen every iteration and it will run from i=0 to n meaning it will run n times. This means that the complexity of the loop itself is O(2n+2) = O(n) since constants and lower order terms are dropped. Since the complexity of a loop is the complexity of the loop itself times, the body that means the loop has a complexity of O(n*1) = O(n). This means the complexity of the while loop's body is O(1+n) = O(n) since lower order terms are dropped. The will loop will run until the linked list is empty and each iteration an elements is removed from it, but whenever a new vertex is found it get added to the linked list, this means that n nodes are added to the linked list across all iterations, meaning the while loop runs n times. Since the while loop runs n times and does the comparison and the body each iteration, meaning the complexity of the while loop is O(n*(n+1)) = O(n^2 + n) = O(n^2) since lower order terms are dropped. After the while loop, there is an if segmens which has a condition that does a comparison and it body does a return, both of which are constant time operations meaning that if segment has a complexity of O(1). Following that there is a return statement which is just O(1). It does not

matter if the if segment is taken or not because either way it the algorithm will only have one more operation left to do which is just a return. This means the complexity of this algorithm is O(1+1+1+n+1+1+1+1+n^2+1) = O(n^2 + n + 8) = O(n^2) since constants and lower order terms are dropped.

The space complexity of this algorithm is O(n) where n is the length of the inputted adjacency matrix. This is because the algorithm first declares an integer variable n, and initializes it to the matrix length using .length, which just returns a value not creating any space. Since that integer variable is only. Declared once, and .length does not make many space, the complexity of it is O(1). After that is an if segment, but it only has a comparison and is not taken since it would leave early resulting in less space being made, which means its complexity is constant O(1). Following that a linked list is created, but no matter how large the matrix is, that linked list will only be made one time, which means it has a complexity of O(1). Then there is an array being created, but the size of the array is equal to the length of the matrix n, which means the complexity of that is O(n), since the space created by the array depends on the length of the matrix. After that an integer count is created, but no matter how large the matrix is it will only be made once meaning it has a complexity of O(1). Then an element is added to the front of the linked list using addFirst, which creates a node and updates pointers meaning that it's space complexity is O(1) since it creates one space. Following that is an array updates and arithmetic which does not create any space meaning its complexity is O(1). Then an integer v is created once no matter how large the matrix is meaning it's complexity is O(1). Then there is a while loop, which has a condition that call on isEmpty, but this method just returns a variable and does a comparison which does not make any space so it's complexity is constant O(1). The loops body first does an update and calls on .remove of a linked list which both don't create space, remove just returns a value and updates pointers so it does not make any space. So the first line is O(1). Then there is a has a body with an if segment, however the if segment's body and condition just does comparisons, updates, arithmetic, and array retrieval which is O(1) in space. However, inside of the if segment there is also an method call to addFirst, which creates a new node and updates pointers for the linked list, which means the complexity of that is O(1) since it will just create one node. However it will only create a new node if that index corresponding to that node has not been visited, which means that the number of nodes created by the for loop can only be n times no matter how many times the for loop or while loop runs. Then the for loop itself creates an integer i only once each of its iterations. This means the while loop has a body of O(1), this does not include the nodes created. Then while loop will run n times because in the worst cases, all nodes will be in the linked list at some point and since it removes one each timer, it will run

n times. This means the space created by the while loop is O(n*1) = O(n) since i is being created n times. After the while loop is a if segment with a return and a return statement, both of those do not create any space so the complexity is constant O(1). This means the space complexity of the algorithm is O(1+1+1+n+1+1+1+1+n+n+1) = O(3n + 8) = O(n) since constants and lower order terms are dropped.

Problem 5:

The time complexity of this algorithm is O(n^4) where n is the length of the matrix. This is because the algorithm first has an if segment that is not taken because it would result in less operations being taken, but the condition is still evaluated. It does two comparisons, and a method call to .length, which just returns a value all of which are constant time operations making the condition have a time complexity of O(1). Following that is a variable declaration and initialization to the matrix's length using .length and an initialization of a variable to false, which all are constant time operations of O(1), which means the complexity of that is O(1). After that there is a for loop structure with a for loop inside of it, another for loop inside of that, and one more for loop inside of that. The innermost loop has two if segments with one inside the other. The innermost one having a condition that does a comparison and array retrieval, and the body does a print which are all constant O(1) time operations. Then, the print statement has nine string concatenations in it, but no matter how large the matrix is, the string will always have fixed length strings being concatenated, and the variables in the concatenations are int's which are fixed at 32 bit values meaning that the largest possible value will only have 10 digits, so no matter how large the input size is there will always be 9 strings being created and 78 characters being transferred meaning it's time complexity will be O(1) since constants are dropped because no matter how large n is it will not be longer than 10 characters. Following that there is a update for cycle to be true, which is just a constant time operation, O(1). This means the inner if segment has a time complexity of O(1) since it's condition and body are constant in time. That if segment is inside of another one, which has a condition that has 4 comparisons and array retrievals, which are all constant time operations and mean the if condition is O(1). That means the outer if segment has a complexity of O(1) since its condition and body are both O(1). That if segment is inside of a for loop, which itself creates a variable v4 and initializes it once then does comparisons and increment v4 until it is greater than or equal to n which means the comparisons and increments happen n times. This means the loop itself has a complexity of O(2n+2) = O(n) since constants and lower order terms are dropped. Since the complexity of a loop is the complexity of the loop itself times the complexity of the body it has a complexity of O(n*1) = O(n). That loop is

inside of an if statement which is inside of a for loop. The if segment has three comparisons and array retrievals, all of which are constant time operations O(1), and the body is the for loop which is O(n), thus making the if statement being O(n+1) = O(n) since lower order terms are dropped. Then for loop this is in creates a variable v3 once and initializes it to 0 and does a comparison and increment each iterations until it is greater than or equal to n meaning those happen n times. This gives them a time complexity of O(2n+2) = O(n) since constants and lower order terms are dropped. Since the complexity of the loop time the body it has a complexity of O(n * n) = O(n^2). The if segment is only taken if v3 is not equal to v2 or v1, which are also for loop values, which means it will only be taken n-2 times each iteration, but in asymptotic complexity is still just n since lower order terms and constants are dropped. That for loop is inside of another if statement which does two comparisons and array retrievals, all of which are constant time complexity making the condition O(1). The complexity of the if section has a whole is then O(1+n^2) = O(n^2) since lower order terms are dropped. That is inside of a for loop, which initializes a variable v2 once to 0 and then does a comparison and increment until it is greater than or equal ton meaning it will do them n times. This makes the loop itself have a complexity of O(2n+2) = O(n) since constants and lower order terms are dropped. This complexity of the loop is the complexity of the loop itself times the body, which means it is O(n*n^2) = O(n^3). That for loop is inside of another one that initializes and declares a variables v1 to 0 and does an increment and comparison until it is greater than or equal to n meaning it does them n times. This means the loop itself has a complexity of O(2n+2) = O(n) since constants and lower order terms are dropped. Since the complexity of a loop is the complexity of the loop times the body, it has a complexity of O(n*n^3) = O(n^4), which is the complexity of the entire nested loop structure. Following that is an if segment which has a comparison and a print statement, which are constant time operations being O(1). This means the complexity of the algorithm is O(1+1 +1+ n^4) = O(n^4 + 3) = O(n^4) since constants and lower order terms are dropped.

The space complexity of this algorithm is O(n^4) where n is the length of the matrix. This is because algorithm first has an if segment, which just has comparisons, and a method call to .length, which does not create any space just returns a variable, meaning this condition makes no space and is O(1). The segment is never taken since it results in less space being created, so the if segment has a complexity of O(1). After that two variables are created, but those variables are only created once no matter how large the matrix is making them be O(1) over all, and there is also a method call to .length but it is constant as previously explained. Then there is a nested loop structure composed of four loops. The inner loop has a body with an if statement nested inside of another one. The inner most if statement

has a body that does a variable update and a print, which are themselves O(1) since they do not create any space. In the print statement however, there are nine string concatenations happening, which means that nine strings will be name, however, no matter how large the input size is, the print will always make nine strings, which means It's complexity if O(9) = O(1) since constants are dropped. Then the inner if has a condition with a comparison and array retrievals, which are constant in space since they do not make any space. This means the inner if has a complexity of O(1) since its condition and body are O91) and creates space. Then the outer if segment has a condition with four comparisons and array retrievals which are constant in space which means that if segment has a complexity of O(1) since it conditions and body are O(1). That makes up the body of the inner most for loop, which creates an integer v4 once and will iterate n times since it will increment v4 until it is greater than or equal to n. This means that the space made in the body will happen n times, but the if segment is only taken n-3 times in the worst case since it happens when it is not equal to v3, v2 or v1. This means the space complexity of the inner for loop is O(n-4+1)= O(n-3) = O(n) since constants and lower order terms are dropped. Then the next outer loop following that is the same as the previous loop, but instead this loop crates a variable v3 and iterates until that is greater than or equal to n resulting in n iterations. That loop has an if statement with the previous one as the body, but the condition just has three comparisons and array retrievals which are constant. Furthermore, that if statement is only taken n-2 times since it will only happen when v3 is not equal to v2 or v1. This means the complexity of the loop is O(1+n(n)) = O(1+n^2) = O(n^2) since constants and lower order terms are dropped. That loop is inside another if statement with a condition of two comparisons and array retrievals which are constant in space meaning that if section has a complexity of O(n^2 + 1) = O(n^2) since constants and lower order terms are dropped. Than is section is inside of another loop, which is the same as the previous loop but this time initializes a variable v2 and increment sit until it is greater than or equal to n, so its body will happen n times. Because of the if condition being v2 cannot equal v1 this if section will only be taken n-1 times, which means the complexity of the loop as a whole is O(1+n(n^2)) = O(1+ n^3) = O(n^3) since lower order terms are dropped. Then that loop is inside of another loop that is the same but this. Time initializes a variable v1 and increments it until it is greater than or equal to n. This means the complexity of the loop is O(1+n(n^3)) = O(1 + n^4) = O(n^4) since lower order terms are dropped. This means the complexity of that nested loop structure is O(n^4). After that is an if section that just does a comparison and a print which are constant in space being O(1). This means the algorithm has a space complexity of O(1+1+1+n^4) = O(3+n^4) = O(n^4) since constants and lower order terms are dropped.

Problem 6:

The time complexity of this algorithm is O(n^2 + k^2+s), where n is the number of vertices, and k is the length of the longest vertex name, and s is the length of the input string.This is because the algorithm first has an if statement that is not taken since it would exit early resulting in less operations being done. The condition is still evaluated, which itself has three comparisons, and method calls to .length() and .charAt(), which just consist of constant time operations .length() just returning a member variable and .charAt() doing an array retrieval, which are O(1). This means the complexity of that condition is O(1) since it only has constant time operations. Following that, an integer variable initialized, and that is just a constant time operation being O(1). Then after that there is a for loop, with a body that has an if statement. The if statement has a condition with one comparison and a call to .charAt, and its body just does arithmetic and an update. All these operations are constant time operations, which means the complexity of the if statement is O(1). The for loop initializes a variable i to 0 once, and then compares it to the string's length, s, and increments by one each iteration until i is greater than or equal to the string's length meaning those will happen s times. This means the complexity of the for loop itself if O(2s+2) = O(s) since constants and lower order terms are dropped. Since the complexity of a loop is the complexity of the loop itself time the body's complexity, it would be O(s * 1) = O(s). After that is a if statement that will make a comparison and call to .length() which are constant time operations, making it have a complexity of O(1) overall. This is not taken since it results in less operations being done. After that is an integer n being initialized to the incremented value in the previous loop, but since that loop counts spaces and a space separates every character, then that +1 is equal to the total number of vertices, thus n is the number of vertices, and even so that has a complexity of O(1) since that initialization is constant. Following that an array of string, is initialized to be of size n, which means n spaces were made and n slots were initilized in the array to default values. Since n slots are initilized, the array has a complexity of O(n). Following that, two variables are initilized, which are constant operations being O(1) overall. Following that is a for loop, with a body that has an if-else condition, but in the worst case resulting in the most operations the else segment is taken the fewest amount of time like once. The if-else segment first starts with a comparison and a method call to .charAt which are constant, but the else body does a variable updates and arithmetic which are also constant time operations. The body of the if section, however, does a string concatenation, which in java requires making a new string and transferring all elements from the first and second string to the new one. This means the complexity of the loop must be determined across all iterations since the number of characters transferred will vary from iteration. The loop runs from i=0 until it is greater than or equal to s, which means it will run s times so in the worst case, the loop will have to

concatenate to build the longest string of size k from the input string. This means across all iterations it would have transferred 1, 2, 3, ..., k times at the end, resulting in the total transfers being 1+2+3+..+k. This follows the sum of positive integers, which is the summation from i=0 to k of i, which equals O(k(k+1)/2) = O(k^2+k/2) = O(k^2) since constants and lower order terms are dropped. This means the complexity of the whole loop is O(k^2) since that is the total numebr of transfers in the worst case. If there are other vertex names if they are the same size as k then the complexity just becomes a constant times k^2 which is still just k^2 since constants are dropped in asymptotic complexity. Additionally, if there are other names shorter than k, then k^2 is an upper bound that none of them can exceed. After that are three variable updates and initializations, all of which are constant time operations being O(1). Then after that a matrix is initialized to be n by n, which means it must initialize n *n spaces making it have a complexity of O(n^2). After that is another for loop, but this one has a body of arithmetic and updates which are just constant time operations, making the body O(1) overall. The loop initializes i once and increments it and compares it to n each time until it is greater than or equal to n, meaning it runs n times. This means the loop has a complexity of O(2n+2) = O(n) since constants and lower order terms are dropped. Since the complexity of a loop itself is complexity of the loop itself times the body, it has a complexity of O(n*1) = O(n). Following that is another loop that runs from i=0 until it is greater than or equal to n while incrementing and comparing i meaning it runs n times and has a complexity of O(n). The loop's body does a print and array retrieval which are constant time operations meaning the body has a complexity of O(1). The complexity of the loop is the body times the loop itself, so this has a complexity of O(n*1) = O(n). After that is a print statement which is O(1). Lastly, there is a for loop with another for loop inside of it. The inner for loop has a body with just prints and array retrievals, which are constant and is overall O(1). The inner loop runs from j=0 to n meaning it runs n times, and thus the complexity of that loop is O(n*1) = O(n). Then the outer loop has a body with the inner loop, an array retrieval, and two print statements which The outer loop itself runs from i=0 to meaning it runs n times and thus has a complexity of O(n). This means the complexity of the loop is O(n*(1+1+n+1)) = O(n+n+n^2+n) = O(3n + n^2) = O(nn^2) since constants and lower order terms are dropped. This means the complexity of the whole algorithm is O(1+1+s+1+1+n+1+k^2+1+n^2+n+1+n^2) = O(2n^2 + k^2 + s + 2n + 7) = O(n^2 + k^2 + s) since constants and lower order terms are dropped.

The space complexity of this algorithm is O(s + n^2), where s is the length of the input string and n is the number of vertices. This is because the algorithm first has an if condition that is not taken otherwise it would return early and less space would be made. The

condition is still evaluated, but it just has comparisons and method calls to .length() and .charAt(), all do not create any space. .length() just returns the value of a stored variable, and .charAt does an array retrieval, all of which don't make any new space meaning that it has a overall space complexity of O(1). Then an integer is declared but no matter how long the input string is only declared once meaning it also has a constant space complexity, O(1). After that is a for loop, which has a body that only contains a comparison, .charAt method call, variable update, and arithmetic which does not create any space meaning the body has a constant space complexity, furthermore no matter how many time the loop runs this body creates no space. The loop itself however does create a variable i only once no. matter how large the input size grows, which means it also has a constant space complexity being O(1). After that is another if statement with a comparison and method call to .length which are constant in space and this is not taken because it would leave really and result in less space being made. After that an integer n is declared once no matter the input size growth and it represents the number of vertices in the string. Then there is an array being initilized to n, meaning n spaces are created for the array thus giving it a space complexity of O(n) since the size of the array can grow with the input size. After that there are two variables being created but they only happen once no matter how much the input string grows, meaning they overall are O(1). After that there is a for loop, which has a body with comparisons and a method call to .charAt(), updates, and arithmetic which don't make space. There is an if-else condition in the body too but no matter which one is taken there will be a new string being made. The else body updates a string to be empty which makes a new string, and the if body does a string concatenation which also makes a new string. So, the body has a complexity of O(1) and creates a new string each time no matter which path is taken. The loop itself does create an integer i but only once no matter the input size, but the loop runs s times since it goes from i=0 incrementing until i is greater than or equal to s. This means the body will happen s times giving the loop an overall space complexity of O(1+s(1)) = O(1+s) = O(s) since lower order terms are dropped. After that is an update which does not make any space and is thus constant in space being O(1). Following that is two variable declarations, but they only happen once no matter how much the input size grows, making them both O(1) overall. After that a matrix is made of dimensions n * n, which means it must create and allocate n * n spaces to store all the values thus making that deceleration have a complexity of O(n^2), since the size of the matrix depends on the number of vertices and is thus does scale with as the input size grows. After that is another for loop, but this loop's body just does updates and arithmetic, which do not create any space and is thus constant meaning no matter how many times the loop runs no space is made. The loop itself only creates an integer i once no matter how large the input size is, thus making it O(1).After that is another for loop, which had a body of prints and an array retrieval all of which do not make any

space, thus making the body constant, and not matter how many times the loop runs no space will ever be made by it. That loop itself creates in integer i but only once no matter how large the input size grows, which means it's complexity if O(1). Following that is a print statement that does not create any space and is thus constant. Finally there is one last for loop, with another for loop inside of it. The inner for loop has a body that just does prints and array retrievals which do not create any space and is constant meaning no matter how many times this loops runs no space will ever be created. The inner loop itself creates an integer j only once during its meaning it has a complexity of O(1). The outer loop has a body of the inner loop, array retrieval, and prints. The array retrieval and prints don't make any space and are thus constant O(1) but the inner loop does create space being O(1), which makes the body O(1) in space overall. The for loop itself creates a variable i only once no matter how large the input size is, but the loop also runs n times since it starts at i=0 and increments it until i is greater than or equal to n meaning the body happens n times. This means the body's space creation is happening n times. This means the space complexity of the for loop is O(1+n(1)) = O(1+n) = O(n) since lower order terms are dropped. This means the space complexity of the algorithm is O(1+1+1+1+1+n+1+s+1+n^2+n+n+1+n^2) = O(2n^2 + s + 3n + 8) = O(n^2 + s) since constants and lower order terms are dropped.